Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Puebla

TE3002B: Implementación de robótica inteligente (Gpo 501)

Profesor: Dr. Rigoberto Cerino Jiménez

Reto semanal 3. Manchester Robotics

Equipo *I miss NWJNS*

Tania Sofia Arrazola Bello	A01738124
Iñaki Ahedo Madrid	A01738231
Marcos Allen Martínez Cortés	A01737939

20 de abril de 2026

Resumen

En este reto semanal 3 se llevó a cabo el desarrollo de un sistema de navegación de lazo cerrado para el robot Puzzlebot utilizando el framework ROS 2. Se implementó una arquitectura distribuida en nodos que permite la localización autónoma mediante odometría y el seguimiento de trayectorias cuadradas de 2 metros de lado, utilizando mensajes personalizados para la gestión de metas a través del tópico `/goals` y el envío de comandos de velocidad mediante `/cmd_vel`.

Durante el desarrollo se diseñó un generador de trayectorias encargado de publicar de forma secuencial los puntos de destino, evaluando en tiempo real si cada objetivo es alcanzable según la dinámica del robot. Asimismo, se implementó un controlador PID robusto para regular tanto la velocidad lineal como el desplazamiento angular. Este controlador fue sintonizado para compensar perturbaciones, ruidos en los sensores y no linealidades del sistema, asegurando que el robot minimice el error de posición en cada nodo de la trayectoria. Finalmente, se consideraron aspectos críticos como la integración de la cinemática diferencial para la estimación de la pose `(x, y, θ)` y la implementación de estrategias de anti-windup en el control integral.

Objetivos

El objetivo principal es desarrollar un sistema de navegación multipunto de lazo cerrado para el Puzzlebot mediante el uso de ROS 2, implementando un controlador PID y un sistema de localización por odometría que permita al robot seguir una trayectoria cuadrada de 2 metros por lado con alta precisión y robustez. Entre los objetivos principales destacan:

- Implementar un nodo de localización que estime en tiempo real la pose del robot (x, y,θ) mediante la integración de la cinemática diferencial y los datos de los encoders.
- Diseñar un generador de trayectorias que publique de forma secuencial los puntos de destino necesarios para completar un cuadrado de 2 metros de lado.
- Desarrollar un controlador de lazo cerrado tipo PID para regular la velocidad lineal y angular, asegurando el seguimiento preciso de los objetivos.
- Implementar estrategias de robustez en el controlador para mitigar los efectos del ruido en los sensores, perturbaciones externas y no linealidades del sistema.
- Utilizar mensajes personalizados para la gestión de metas a través del tópico `/goals`, incluyendo la validación de puntos alcanzables por el robot.
- Publicar comandos de velocidad calculados por el PID en el tópico `cmd_vel` mediante mensajes de tipo Twist.
- Sintonizar los parámetros de ganancia (`Kp, Ki, Kd`) del controlador para optimizar el tiempo de respuesta y minimizar el error en estado estacionario.

Introducción

En este reto se aborda la navegación autónoma de un robot móvil utilizando ROS 2, mediante la implementación de un sistema de control en lazo cerrado y localización por odometría. El sistema permite que el robot siga una trayectoria cuadrada de 2 metros de lado, utilizando un controlador PID para corregir su movimiento en tiempo real a partir de la estimación de su pose. Para lograrlo, se emplea la integración de la cinemática diferencial y el uso de mensajes personalizados para la gestión secuencial de metas. Este enfoque permite comprender el comportamiento de un robot móvil bajo un esquema de retroalimentación, mejorando la precisión y robustez ante perturbaciones y ruidos del entorno en comparación con el control en lazo abierto.

Odometría de un robot móvil

La odometría es el estudio de la estimación de la posición y el trayecto de vehículos o robots móviles mediante datos de sensores de movimiento, su nombre proviene del griego hodos ("viajar", "trayecto") y metron ("medida"), y se basa en la integración de información incremental para calcular cambios respecto a un punto de inicio (Blade, 2024; Ramírez Benavides, s. f.). Existen diferentes métodos de medición entre los que se encuentran los encoders de las ruedas, las unidades de medición inercial (IMUs) y la odometría óptica y visual (Blade, 2024).

La odometría destaca por su bajo costo, alta velocidad de muestreo y buena precisión a corto plazo, aunque presenta desafíos críticos como la acumulación de errores, estimación de una posición relativa y no absoluta, y sensibilidad a factores externos como el deslizamiento de las ruedas; debido a esto, se utiliza la fusión de sensores combinando diferentes métodos, aunque esto requiere algoritmos más complejos y mayor potencia de procesamiento (Blade, 2024; Ramírez Benavides, s. f.).

Un robot móvil diferencial se caracteriza por poseer dos ruedas unidas por un eje transversal, como se muestra en la figura 1, donde cada rueda es accionada de forma independiente por su propio motor y monitoreada por un encoder que registra las revoluciones del eje (Ramírez Benavides, s. f.). Esta configuración permite el desplazamiento y la rotación del sistema mediante la diferencia de velocidades de ambas ruedas. Para estimar la posición del robot en el plano, se recurre al Método de Euler, el cual permite transformar las ecuaciones diferenciales de movimiento en un problema de condiciones iniciales (IVP) donde la posición actual $[x_n, y_n, \theta_n]$ actúa como el punto de partida para predecir el siguiente estado (Dawkins, 2022). Al aplicar la aproximación de Euler, se asume que las velocidades lineal y angular permanecen constantes durante un intervalo de tiempo pequeño *h* o *dt*, permitiendo calcular la nueva postura como $y_{n+1} = y_n + h \cdot f_n$.

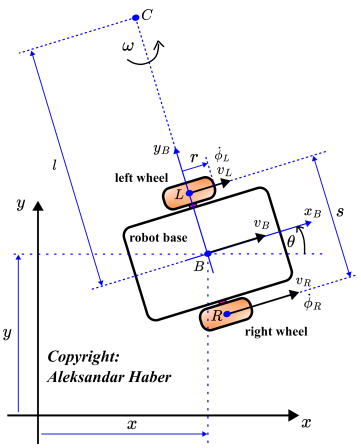

Figura 1: Robot móvil diferencial.
https://aleksandarhaber.com/wp-content/uploads/2023/10/robot_dd2.png

Mensajes 'Odometry'

Los mensajes de odometría (`nav_msgs/msg/Odometry`) son la interfaz estándar para estimar la posición y velocidad de un robot en el espacio libre. Se compone de cuatro campos base (Howard, s. f.; *Odometry,* s. f.):

- `header`: contiene la marca de tiempo y el ID del sistema de coordenadas de referencia (padre).
- `child_frame_id`: el sistema de coordenadas asociado al robot (hijo); las velocidades se expresan respecto a este.
- `pose`: contiene la posición y orientación relativa a un marco fijo (mundo). Incluye una covarianza (matriz de incertidumbre de 36 elementos).
- `twist`: contiene las velocidades lineales y angulares.

Dentro del campo /`pose`, la información se divide en dos subcampos (Howard, s. f.):

- Posición (`Point`): define las coordenadas x, y, z del robot en metros.
- Orientación (`Quaternion`): define hacia dónde apunta el robot mediante cuaterniones (x, y, z, w), que son la forma estándar en ROS 2 para la orientación porque ofrecen una solución robusta y unificada para la cinemática de robots en un espacio tridimensional.

Para mover al robot y alterar su pose, se utiliza el tópico `/cmd_vel` con el tipo de mensaje `geometry_msgs/msg/Twist`; así, este mensaje permite controlar los seis grados de libertad de un robot (Howard, s. f.).

Cálculo del error en un robot móvil diferencial

En el cálculo de la odometría, el error se manifiesta principalmente en dos componentes: el error lineal y el error angular, los cuales se derivan de las discrepancias entre el movimiento real y la interpretación de los datos de los encoders. El error lineal se asocia frecuentemente con el parámetro E_d (diámetro de las ruedas), donde una

diferencia entre los diámetros de la rueda derecha D_R y la izquierda D_L, provoca que el robot recorra una distancia distinta a la calculada; matemáticamente este se estima comparando la posición absoluta final (x_{abs}, y_{abs}), contra la posición calculada (x_{calc}, y_{calc}) tras completar una trayectoria. Por otro lado, el error angular es sumamente crítico debido a que pequeñas desviaciones en la orientación se magnifican con la distancia recorrida; este error suele vincularse al parámetro E_b, que representa la incertidumbre en la distancia entre ejes del robot (Ramírez Benavides, s. f.).

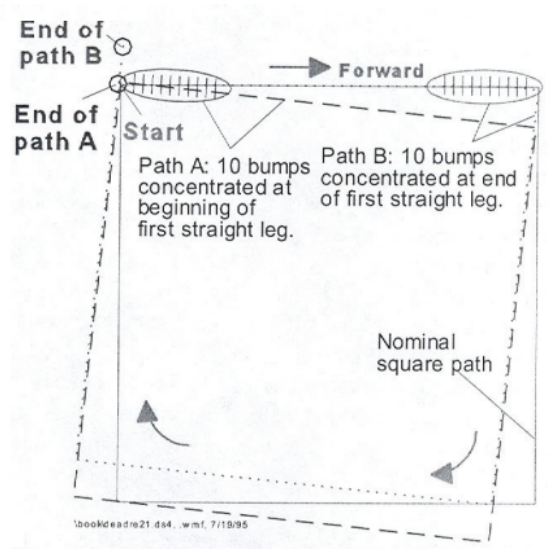

Figura 2: Prueba UMBmark (Ramírez Benavides, s. f.).

Para calcular estos errores de manera efectiva, se utiliza la diferencia de retorno en pruebas de trayectoria cerrada, como la prueba UMBmark que se puede observar en la figura 2. En estas pruebas el error lineal se cuantifica mediante el desplazamiento en los ejes x e y ($\epsilon_x = x_{abs} - x_{calc}$ y $\epsilon_y = y_{abs} - y_{calc}$), mientras que el error angular se determina a través del error de orientación en retorno ($\epsilon_\theta = \theta_{abs} - \theta_{calc}$). Mientras que el error lineal puede ser causado por el deslizamiento o diámetros desiguales, el angular es particularmente sensible a la resolución de los encoders y a la anchura de la base del vehículo. La distinción entre ambos es vital, ya que los errores sistemáticos (relacionados al diseño y mecánica del robot) tienden a hacer que las elipses de incertidumbre, como se observan en la figura 3, crezcan de forma acelerada en la dirección de la marcha (Ramírez Benavides, s. f.).

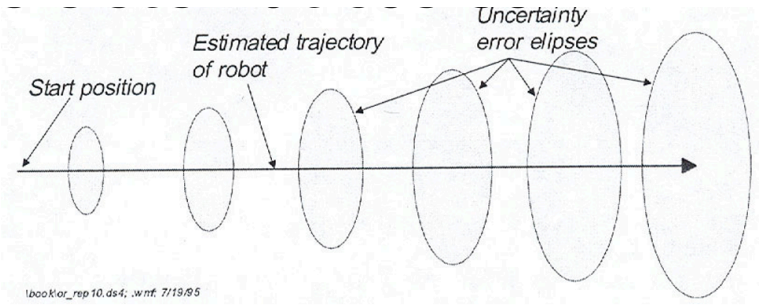

Figura 3: Elipses de incertidumbre (Ramírez Benavides, s. f.).

Control en lazo cerrado para un robot móvil

El sistema opera bajo un esquema de Control en lazo cerrado. A diferencia del lazo abierto, este método utiliza la retroalimentación constante de la pose estimada para comparar la posición actual con la meta deseada. Es decir, que utiliza la información del estado actual del robot para ajustar sus acciones y alcanzar un objetivo deseado [1]. A diferencia del lazo abierto este sistema detecta y corrige errores generador por perturbaciones externas, ruido en los sensores, o incertidumbres del modelo como el deslizamiento de las ruedas.

El funcionamiento se basa en un ciclo continuo que puede clasificarse en estas etapas [1,2,3]:

1. **Estimación y Percepción de la Pose:** El robot utiliza sensores como el IMU para la orientación y determinar su posición real `(x, y, θ)` así como también encoders para la velocidad de las ruedas [1]. La `/pose` o `/goals` en este reto se encarga de determinar la referencia global del robot en el plano a partir de la integración discreta de las velocidades lineales del robot las cuales se determinan por el modelo cinemático directo del robot [1].
2. **Cálculo del error:** Se compara la posición real con la trayectoria de referencia (el camino deseado). La diferencia entre ambas se denomina *error de seguimiento.* El error de seguimiento es la medida de la desviación entre el estado actual del robot y el estado deseado definido por la trayectoria. En este proyecto, el error no se calcula de forma global, sino que se divide en dos componentes polares para facilitar el control del robot diferencial:
    a. *Error de Distancia (Rho)*: Se define como magnitud del vector que une la posición actual del robot (`x,y`) con la coordenada objetivo (`x_goal, y_goal`), esto representando cuanto camino lineal le falta recorrer al robot. La fórmula es la siguiente:

$$\rho = \sqrt{(x_goal - x)^2 + (y_goal - y)^2}$$

    b. *Error de Orientación (Alpha):* Es la diferencia angular entre el ángulo de vista del robot y el ángulo necesario para apuntar directamente hacia el objetivo, este error indica al robot cuánto tiene que girar sobre su eje para lograr el objetivo. La fórmula es la siguiente:

$$\alpha = atan2\,(y_goal - y,\ x_goal - x) - \theta$$

3. **Acción de Control**: Un controlador procesa el error y genera comandos de velocidad para los motores que minimicen la desviación.

PID aplicado a un robot móvil diferencial

La aplicación del controlador en este proyecto se estructura mediante un lazo de control de posición global. A diferencia de los sistemas industriales complejos que controlan cada rueda por separado, este sistema utiliza la retroalimentación de la odometría para corregir el desplazamiento del robot hacia una meta específica en el plano. Para que el Puzzlebot se desplace con precisión, el sistema se enfoca en el siguiente nivel:

Lazo Externo de Seguimiento de Puntos

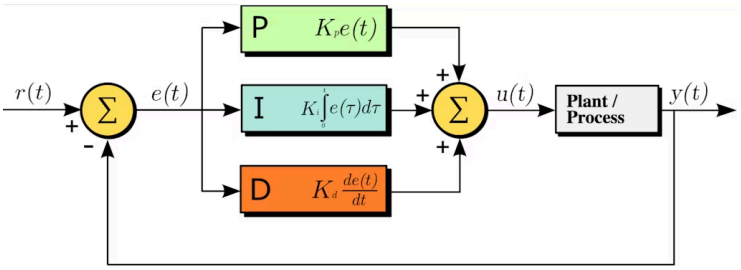

Figura 3. Diseño de un PID https://dewesoft.com/es/blog/que-es-un-controlador-pid

Este lazo utiliza el modelo cinemático del robot para calcular las velocidades lineal (`v`) y angular (`w`) necesarias para alcanzar un punto. Compara la pose real estimada (obtenida mediante el nodo de localización y encoders) con la posición de la meta para generar los comandos de velocidad.

En este caso, el `PID` corrige el error de distancia (ρ) hacia el objetivo:

- **Proporcional (P)**. Reduce el error de distancia actual. Si el robot está lejos del punto, aumenta la velocidad lineal proporcionalmente a esa diferencia para acercarse más rápido.
- **Integral (I)**. Elimina el error acumulado. Es fundamental para asegurar que el robot llegue exactamente a la coordenada deseada, compensando la fricción que podría detener al robot antes de tiempo. Se implementó con una técnica de clamping para evitar saturación.
- **Derivativo (D)**. Anticipa cambios rápidos en la distancia y ayuda a suavizar el frenado cuando el robot se acerca a su valor de referencia, evitando que se pase de la meta.

Robustez de un controlador

La robustez de un controlador se basa principalmente en su capacidad para garantizar el cumplimiento de un objetivo de control, seguir una trayectoria, ante la presencia de perturbaciones externas, ruidos en los sensores e incertidumbres físicas no modeladas [1]. Los principales puntos a tomar en cuenta para un sistema robusto son:

1. **Manejo de Incertidumbres y Perturbaciones.** Un controlador robusto debe ser capaz de minimizar el impacto de factores que no están incluidos en el modelo

matemático original, tales como el deslizamiento de las ruedas, no linealidades y ruidos e imperfecciones del entorno [1].

2. **Estrategias de Control Específicas**. Existen algunas técnicas para el diseño que implementan un control robusto como lo son:
    a. *Control en Modo Deslizante (SMC)*: Es una de las técnicas más citadas por su robustez. Se basa en forzar al sistema a permanecer sobre una superficie deslizante predefinida [1]. Una vez que el sistema alcanza este estado, adquiere características invariantes ante perturbaciones.
    b. *Control de Espacios Nulos (Hierarquización)*: Proporciona robustez al establecer una jerarquía de tareas donde la tarea principal, como la posición, se ejecuta sin interferencia de las tareas secundarias, como la orientación, proyectando estas últimas en un espacio que no afecta la precisión del objetivo primordial [1].

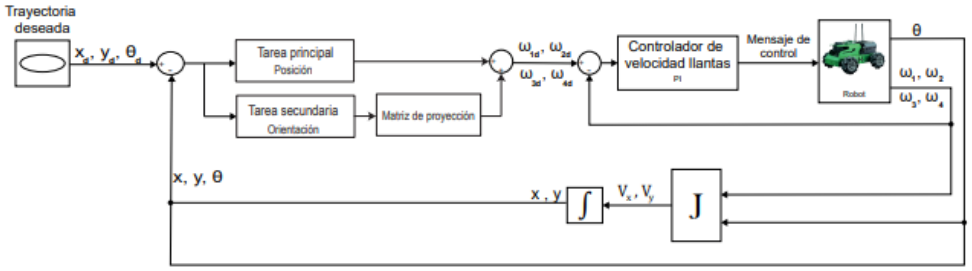

Figura 4. Diseño de un controlador usando lazo de control en espacios nulos [1]

3. **Arquitectura de Lazo Cerrado y Sintonización.** Para que un controlador, como el `PID`, sea robusto, sus ganancias deben ajustarse cuidadosamente considerando las perturbaciones y ruidos específicos del entorno de operación. Además, para mejorar la robustez frente a sensores ruidosos, se utilizan observadores, como los basados en la técnica *Super Twisting*, que permiten obtener estimaciones de velocidad más limpias y estables que la simple derivación de la posición [2]. Por último, la implementación de una estructura de doble lazo, con un lazo interno de velocidad, asegura que las ruedas mantengan la consigna deseada incluso ante obstáculos físicos como pendientes, haciendo que el sistema sea resistente a fallos mecánicos menores [1].

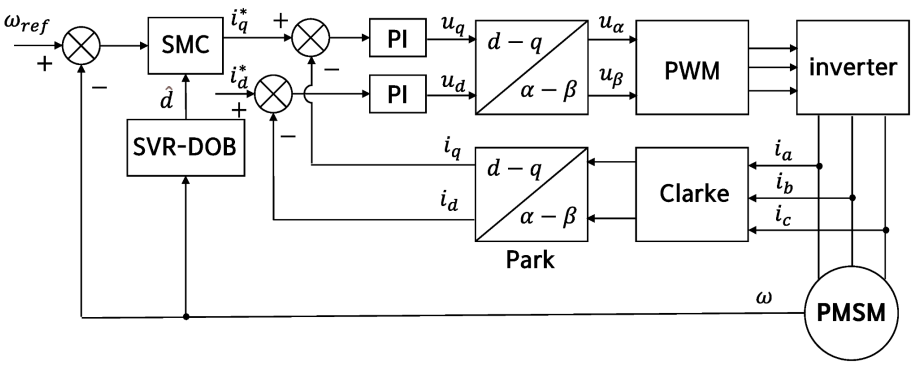

Figura 5. Diseño de un controlador usando la técnica super twisting, PI y SMC para mejorar la robustez https://www.mdpi.com/2076-3417/12/21/10749

Cinemática de robots con accionamiento diferencial

Es la representación matemática que describe cómo el movimiento de sus dos ruedas independientes se traduce en el desplazamiento y la rotación del vehículo [4]. En el caso del Puzzlebot, se utiliza un sistema de accionamiento diferencial, el cual consiste en dos ruedas motrices independientes situadas en un mismo eje y una rueda loca para brindar estabilidad. Este modelo relaciona las velocidades angulares de cada rueda con la velocidad lineal y angular total del robot basándose en su geometría, específicamente en el radio de las ruedas (`r`) y la distancia entre ellas (`L`) [4]. El funcionamiento de este tipo de robots se divide en dos enfoques principales:

- **Cinemática Directa (Forward Kinematics)**: Permite calcular la velocidad lineal v_x y la velocidad angular w del robot a partir de las velocidades de rotación medidas de la rueda derecha y de la rueda izquierda
- **Cinemática Inversa (Inverse Kinematics):** Se utiliza para determinar qué velocidades deben tener las ruedas para que el robot alcance una velocidad lineal y angular deseada

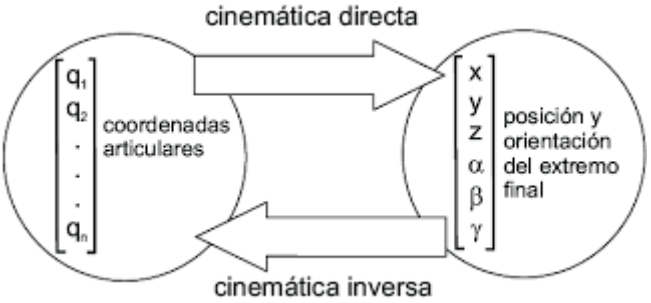

Figura 6. Relacion entre la cinematica directa e inversa https://www.researchgate.net/figure/Figura-216-Relacion-entre-cinematica-directa-e-inversa_fig15_287995531

El modelo se divide en dos partes:

1. *Cálculo de velocidades lineales y angulares.* La velocidad lineal total v del centro del robot y su velocidad angular w se obtienen mediante las siguientes ecuaciones:

$$\omega = (v_R - v_L)/b \qquad V = \omega \cdot R = \frac{v_R + v_L}{2} \qquad (1)$$

   Donde b es el ancho de la via o distancia entre las ruedas (wheelbase) [5].

2. *Representación en coordenadas globales.* Para conocer la posición del robot en cada instante, se transforma el movimiento del cuerpo al sistema de coordenadas global utilizando el ángulo de orientación actual ϕ [5]

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\varphi} \end{bmatrix} = \begin{bmatrix} \cos\varphi & 0 \\ \sin\varphi & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} V \\ \omega \end{bmatrix} \tag{2}$$

Aplicación en el controlador

En la implementación de este reto, se enfrenta el problema de la cinemática inversa: dado que el controlador PID genera una velocidad lineal v y una angular w Para alcanzar una meta, es necesario calcular la velocidad que debe tener cada rueda. Esto se hace mediante las ecuaciones:

$$\begin{aligned} v_R &= r \cdot \omega_R \\ v_L &= r \cdot \omega_L \end{aligned} \tag{3}$$

$$\omega_R = \frac{V + \omega \cdot b/2}{r} \qquad \omega_L = \frac{V - \omega \cdot b/2}{r} \tag{4,5}$$

Las velocidades se convierte a radianes por segundo (w_{rueda}) dividiendo entre el radio de la rueda (r).

Solución del problema

1. **Estimación de pose**

En este reto, la posición del robot ya no se estimaba dependiendo del tiempo de ejecución dada ya sea la velocidad o tiempo en el que debía hacer la trayectoria, sino que se implementó un nodo de localización. Este es el encargado de recibir la retroalimentación de los encoders para resolver el problema de la odometría.

Se configuraron dos suscriptores a los tópicos `VelocityEncr` y `VelocityEncL` los cuales son tópicos propios del Puzzlebot que publican información de los encoders. En este nodo también se utilizó una política de `QoS` (Quality of Service) de tipo `Best Effort` que garantiza recibir los datos aunque haya latencia en la que se pueden perder los paquetes.

Se utilizaron las constantes físicas del Puzzlebot como el radio de la rueda $r = 0.05m$ y la distancia entre ruedas $l = 0.18m$ las cuales se implementaron en las ecuaciones de espacio de estado para transformar las velocidades angulares de las ruedas en velocidades lineales y angulares del chasis.

$$v = r\frac{\omega R + \omega L}{2}, \ \omega = r\frac{\omega R - \omega L}{l}$$

Después, se necesitó utilizar la integración de Euler con un diferencial de tiempo Δt que se calculaba constantemente mediante el reloj del sistema de ROS 2. Esto permite obtener a partir de la velocidad, la estimación de la pose (x, y, Θ). Al usar el reloj de ROS 2, permite que la estimación sea mejor a pesar de que haya variaciones del procesamiento del CPU.

Para cumplir con el estándar del tipo de mensajes que maneja ROS 2 para los datos de odometría que son de tipo `nav_msgs`, la orientación se convirtió de ángulos de Euler a cuaterniones con la librería de `transforms3d`, publicando el mensaje completo al tópico /odom.

2. **Generador de trayectorias con retroalimentación**

El nodo generador de trayectorias a diferencia del reto anterior, este se encarga de coordinar la progresión de los puntos a los que debe llegar el robot o waypoints mediante el monitoreo constante de la posición suscribiendote al tópico de /odom del nodo de localización.

Primero se definió una trayectoria de forma de cuadrado de `2 x 2` metros. Los vértices se establecieron con las coordenadas `(2, 0)`, `(2, 2)`, `(0, 2)` y `(0, 0)` el cual asegura que el robot regrese a su posición original.

Después de definir la trayectoria, el nodo calcula constantemente el error de posición actual, definido como la distancia euclidiana (ρ) entre la ubicación del robot y el objetivo. Para optimizar el movimiento y evitar que el robot se detenga, se implementó un umbral de aceptación de `0.05` metros para los puntos intermedios. Una vez que el robot ha ingresado en ese rango de tolerancia, el sistema actualiza el índice del objetivo y publica el siguiente destino.

La información que se le transfiere al nodo controlador se realiza mediante un mensaje personalizado llamado `GoalList`. Este mensaje tiene la estructura para que el control a lazo cerrado espere la información de la siguiente manera:

`current_goal` es el que especifica la coordenada exacta hacia la cual el controlador debe dirigir los esfuerzos del motor y orientación.

`next_goal` es el que proporciona la ubicación del siguiente vértice, permitiendo al sistema tener cambios de dirección.

`final_goal` usa una bandera booleana que se activa solo al procesar el último vértice de la trayectoria, indicando al controlador que debe detener la trayectoria de forma inmediata para evitar errores acomulativos que afecten el control.

3. **Controlador de lazo cerrado usando PID**

Sustituyendo el sistema a lazo abierto del reto pasado el cual se utilizaban máquinas de estado basadas en tiempo, este controlador implementa un algoritmo que es reactivo ante las lecturas de los encoders. Esto permite que el robot tenga una respuesta inmediata ante su posición real en el espacio.

Primero para que la navegación esté optimizada, el controlador hace una conversión de coordenadas cartesianas que son las que provienen del nodo que publica la odometría, hacia un sistema de coordenadas polares que es relativo al punto de destino.

En este caso ρ es la hipotenusa del triángulo formado por la distancia en `X` y `Y`. Representa la distancia lineal que le falta al robot para llegar al punto objetivo. Este se calcula de la siguiente manera:

$$\rho = \sqrt{\Delta x^2} + \Delta y^2$$

Después se calcula α que es el error de orientación. Primero se calcula el ángulo hacia el objetivo usando `arctan2` y luego se resta la orientación actual del robot. Este resultado es cuánto debe girar el robot.

$$\alpha = atan2(\Delta y, \Delta x) - \Theta$$

Se implementó un controlador `PID` para la velocidad lineal. Se creó una clase para el control `PID` que procesa el error de distancia ρ. El controlador genera una velocidad lineal proporcional a la proximidad del objetivo, integrando las tres componentes del `PID`.

4. Implementación y Sintonización

Los pasos técnicos seguidos en este reto son:

1. **Cálculo Cinemático:** Se utiliza la cinemática diferencial para estimar la posición del robot y determinar el ángulo hacia la meta.
2. **Lectura de Sensores**: Se utilizan los encoders del Puzzlebot para obtener la velocidad de cada rueda, la cual se integra en el nodo de localización para conocer la posición actual (`x, y`).
3. **Sintonización de Parámetros**: No se utilizó Ziegler-Nichols. Las constantes del controlador (`Kp, Ki, Kd`) se determinaron mediante un método experimental de prueba y error, ajustando los valores hasta obtener un movimiento estable y preciso.
4. **Discretización:** El controlador se implementó de forma digital en un nodo de Python dentro de ROS 2, calculando el error y la salida en cada paso de tiempo (`dt`) medido por el reloj del sistema.

Para corregir el error de orientación α, se aplicó un simple control proporcional junto con la función `wrap_to_pi` la cual restringe las rotaciones a un rango de

$-\pi, \pi$ para asegurarse de que los giros sean más óptimos y no dar vueltas innecesarias. Este control asegura que tenga correcciones constantes lo que permite que el avance lineal coincida con el vector que se definió como waypoint.

Finalmente, aplicando el control PID para la velocidad lineal y el control P para la rotación, la lógica de control es la siguiente:

1. SI el error angular hacia el objetivo es mayor de `0.1 rad`, el sistema prioriza la rotación
2. Una vez que está alienado, el control PID es el que se encarga de la velocidad lineal mientras que el control angular realiza algunos ajustes mínimos de manera simultánea

Una vez que la distancia es menor al umbral de tolerancia que está definido como `0.05m`, el error de posición se considera que ha sido mitigado. Cuando se cumple esta condición, el sistema valida la llegada del robot al destino y el generador de trayectorias procede a publicar el siguiente punto.

5. Manejo de no-linealidades

Para que el controlador `PID` fuera lo más preciso posible, se aplicaron algunas estrategias para que el sistema fuera lo más robusto posible.

Se utilizaron valores de zona muerta y saturación $v_{min} = 0.009m/s$ y $v_{max} = 0.2m/s$ para limitar la salida del `PID` evitando que el controlador demande velocidades que supere la velocidad máxima a la que puede alcanzar o velocidades que no superan la inercia inicial.

Para los puntos intermedios de la trayectoria cuadrada que debe realizar el robot, se estableció una tolerancia dinámica la cual es de `5 cm` para permitir un movimiento fluido sin que se detenga totalmente en cada esquina, mientras que para el punto final es de `3 cm` para que pueda cerrar de manera exacta la trayectoria.

Por último, el proceso de sintonización del controlador `PID` se realizó mediante el método heurístico:

1. Se incrementó `Kp` hasta observar una respuesta deseada y rápida
2. Se añadió `Kd` para suavizar las detenciones en las esquinas y eliminar oscilaciones
3. Se ajustó `Ki` mínimamente para corregir desvíos laterales constantes
4. Se verificó que el robot fuera capaz de completar un cuadrado de 2 x 2 metros regresando al punto `(0, 0)` con la mejor precisión posible evitando que llegara antes o después a alguno de los 4 puntos y también evitando que en línea recta tuviera ligeras desviaciones.

6. Estrategias de robustez implementadas

Para asegurar la robustez del Puzzlebot en este challenge, se aplicaron las siguientes estrategias dentro del código:

1. *Anti-Windup/Clamping*. En el término integral del PID, se implementó una técnica de limitación o clamping. Esto evita que el error acumulado crezca de forma desmedida cuando el robot encuentra un obstáculo físico o una saturación en los motores, previniendo oscilaciones violentas al recuperar el movimiento.
2. *Manejo de Zonas Muertas (Deadzone)*. El controlador incluye un límite mínimo de salida (`min_out`). Esto asegura que el voltaje enviado a los motores sea siempre suficiente para vencer la fricción estática, permitiendo que el robot se mueva incluso ante errores muy pequeños.
3. *Filtrado de Orientación con wrap_to_pi.* Para evitar giros innecesarios o bruscos, se utilizó una función que normaliza el error angular entre $-\pi, \pi$. Esto garantiza que el controlador siempre elija la ruta de giro más corta, aportando estabilidad a la navegación.
4. *Tolerancia de Error (Threshold).* Se estableció un umbral de precisión de `5 cm` para considerar una meta como alcanzada. Esta tolerancia permite que el sistema sea inmune al ruido de baja amplitud de los encoders, evitando que el robot realice micro-correcciones infinitas en el punto de destino.

Resultados

Se realizaron distintas pruebas del sistema utilizando el Puzzlebot con el objetivo de validar el correcto funcionamiento de este. Inicialmente se probó realizando una trayectoria en forma de cuadrado de `2 x 2` metros la cual estaba definida en el nodo de `path_generator` como una lista de objetivos de la siguiente manera: `[[2.0, 0.0],[2.0, 2.0],[0.0, 2.0],[0.0,0.0]]`.

De igual manera se implementó la visualización gráfica de las trayectorias en tiempo real mientras el robot se movía. Esto funcionó para saber qué tan exacto seguía la trayectoria y si se tenía que modificar algún valor del controlador `PID`.

Teniendo este sistema de interacción con el usuario, se establecieron las siguientes trayectorias:

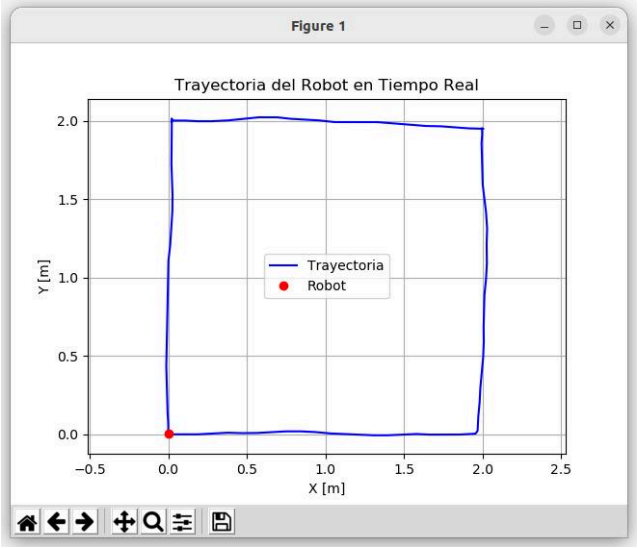

Fig 7. Gráfica de la trayectoria del cuadrado

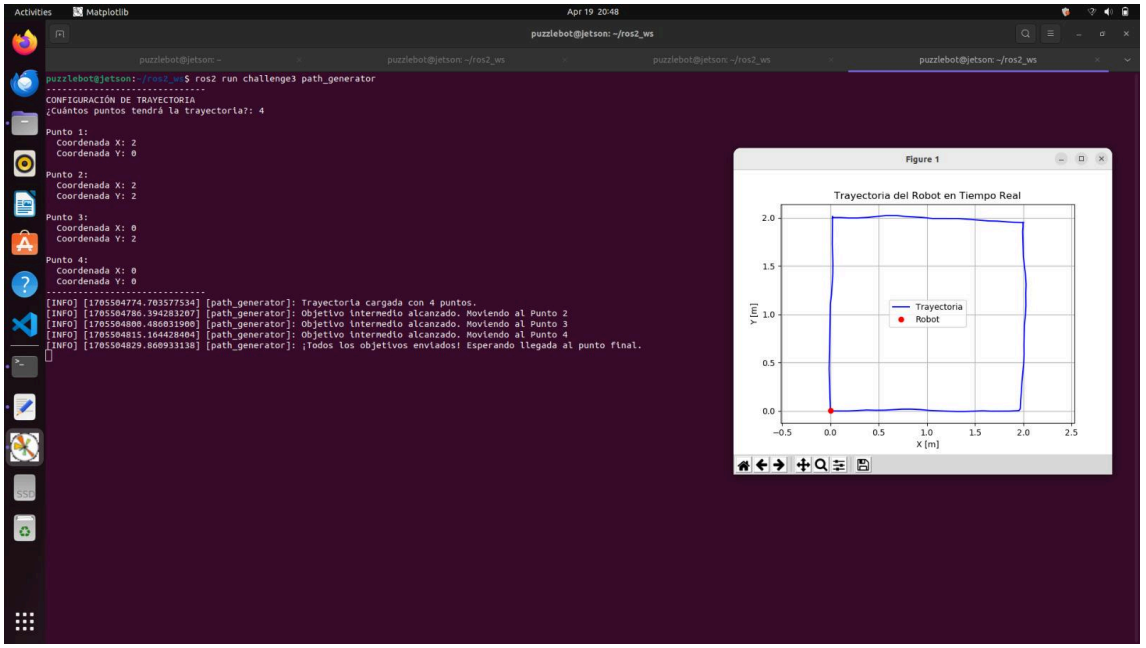

Fig 8. Terminal que muestra los puntos establecidos por el usuario

1. **Cuadrado.** La primera fase de las pruebas consistió en la ejecución de una trayectoria en forma de cuadrado, definida por la secuencia de coordenadas: `[[2.0, 0.0], [2.0, 2.0], [0.0, 2.0], [0.0, 0.0]]`. Como se aprecia en la gráfica de la *Figura 8*, el robot logró seguir el camino de referencia con un alto nivel de precisión. Durante el desplazamiento, se observó que el Puzzlebot ejecuta los giros de 90 grados con una respuesta dinámica veloz y estable. Este comportamiento es resultado directo de la implementación del término derivativo en el controlador PID, el cual permite amortiguar la velocidad angular a medida que el error de orientación se aproxima a cero. Gracias a esta sintonización, se previene el sobre impulso (overshoot) en las

esquinas, permitiendo que el robot alcance el ángulo objetivo con precisión y sin presentar oscilaciones residuales.

Link de video demostrativo: https://drive.google.com/file/d/1OOxoyoQ2Ju7Lvqz3rcpT32QqVmDZV1Fy/view?usp=drive_link

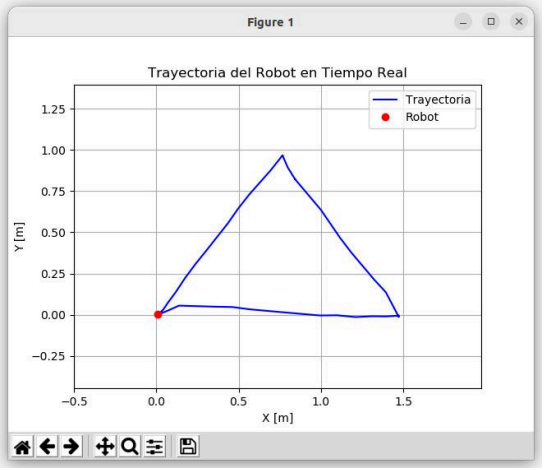

Fig 9. Gráfica de la trayectoria del triángulo

2. **Triángulo.** Para esta fase del challenge, se configuraron tres puntos de consigna o waypoints con las coordenadas: `(1, 0)`, `(0.5, 1)` y `(0, 0)`. El sistema fue diseñado para graficar la trayectoria en tiempo real mientras el nodo generador de trayectorias informaba a través de la terminal la llegada exitosa a cada objetivo. Como se observa en los resultados, el robot completó el recorrido de manera satisfactoria. Un aspecto relevante del desempeño del controlador fue la ejecución de la estrategia de prioridad angular: cuando el error de orientación (α) es superior a 0.1 radianes, el algoritmo establece la velocidad lineal en cero. Esta condición obliga al vehículo a detener su avance para corregir el rumbo girando sobre su propio eje. Solo una vez que el error angular se encuentra dentro del rango de tolerancia, el controlador `PID` lineal se activa para reanudar el desplazamiento hacia el siguiente punto. Esta lógica de control asegura que el robot no se desvíe de la ruta planificada y mantenga la precisión en el seguimiento de trayectorias complejas.

Link del video demostrativo: https://drive.google.com/file/d/1ZyWC86vmLqisLKzd6a2R_HX4bdIiq6W7/view?usp=drive_link

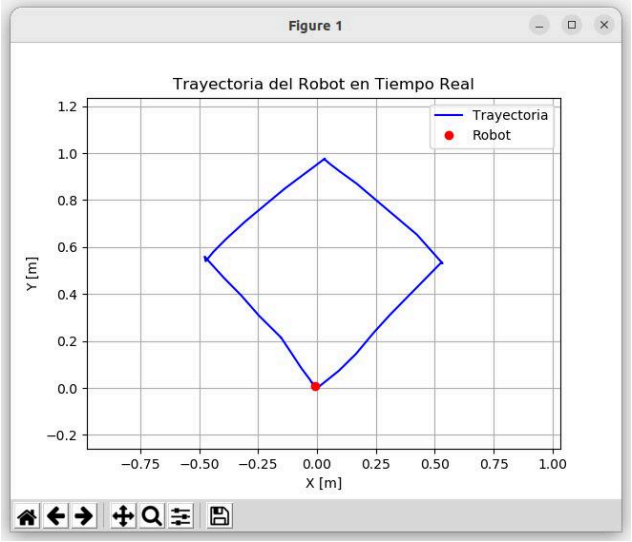

Fig 10. Gráfica de la trayectoria del rombo

3. **Rombo:** En la Figura 10 se presenta la trayectoria generada en tiempo real a partir de la secuencia de puntos de consigna: `(0.5, 0.5)`, `(0, 1)`, `(-0.5, 0.5)` y `(0, 0)`. Los resultados experimentales demuestran que el Puzzlebot cumple con el recorrido de manera satisfactoria, manteniendo un control preciso sobre las velocidades lineal y angular en todo momento. Se destaca la capacidad del sistema para transitar entre coordenadas con cambios de dirección constantes, validando la eficacia del controlador de lazo cerrado y el algoritmo de seguimiento de metas ante trayectorias que exigen maniobras en múltiples cuadrantes del plano.

   Link del video demostrativo:

   https://drive.google.com/file/d/1RlxHWa6hhRJ5qauSbFGvx8aXG8HO_55O/view?usp=drive_link

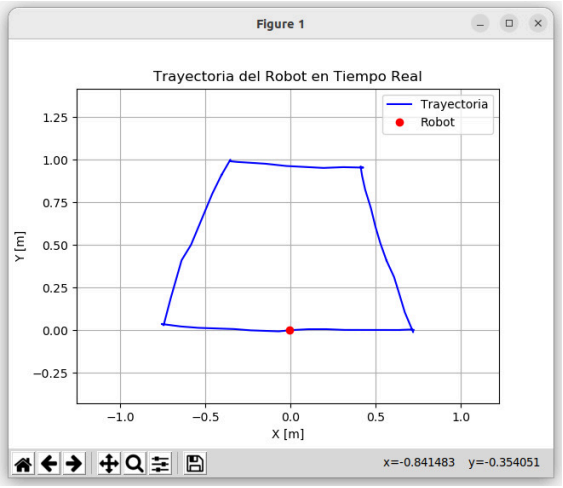

Fig 11. Gráfica de la trayectoria del rapecio

4. **Trapecio.** En la Figura 11 se presenta la trayectoria generada en tiempo real a partir de la secuencia de puntos de consigna: `(0.75, 0.0), (0.5, 1), (-0.4, 1), (-0.75, 0)` y `(0, 0)`. Esta prueba permitió evaluar el desempeño del sistema ante una ruta no convencional que requiere desplazamientos en ángulos agudos y cambios de dirección constantes. Los resultados muestran que el Puzzlebot navega entre los waypoints de manera fluida, validando la precisión del nodo de localización y la robustez del controlador PID.

   Link del video demostrativo: https://drive.google.com/file/d/1ROLYIzjK5DYhMSKs5GurwPIQ0ZyA-v70/view?usp=drive_link

Aquí se muestran los videos con distintas trayectorias

https://drive.google.com/drive/folders/1KZ0wca2FRAz5sro1TaWxma0XOFuekp2_?usp=sharing

Conclusiones

Se cumplieron los objetivos de este reto al implementar un sistema de navegación de lazo cerrado que es capaz de seguir una trayectoria definida por un nodo `path_generator` con una precisión superior a la obtenida en el reto anterior. A diferencia al enfoque que tiene un lazo abierto que solo está basado en tiempo, el uso de retroalimentación constante por parte de los encoders propios del Puzzlebot, permitió que el algoritmo generará comandos de velocidad que reaccionan activamente a esta información mediante controladores `P` y `PID`, eliminando así la dependencia de factores de calibración manuales y adaptándose a las condiciones del entorno. Se logró integrar la localización por odometría a través del mensaje personalizado GoalList, asegurando que el Puzzlebot alcanzará cada vértice dentro de los rangos de tolerancia. Sin embargo, se observó que aunque el controlador es robusto ante variaciones físicas como la fricción, el sistema aún presenta errores en la trayectoria debido a que confía plenamente en las lecturas de los encoder, lo que significa que si las ruedas se deslizan sobre el suelo, el sistema registrará un avance que no hizo, acomulando un error provocando que no siga exactamente la trayectoria. Lo que se puede mejorar es una mejor sintonización del controlador y a su vez utilizar otras referencias externas como las de una cámara para evitar este tipo de perturbaciones que el controlador por retroalimentación de encoders no puede solucionar.

Referencias

[1] López Doicela, R. S. (2025). Estrategias de control, estimación y evasión de obstáculos para un robot móvil omnidireccional con ruedas Mecanum: Desarrollo de un controlador cinemático de seguimiento de trayectorias [Trabajo de Integración Curricular, Escuela Politécnica Nacional]. Repositorio Institucional EPN.

[2] Sáenz, A., Santibáñez, V., Bugarín, E., Dzul, A., & Ríos, H. (2017). Control de velocidad por voltaje calculado de un robot móvil omnidireccional con realimentación visual. En Memorias del XIX Congreso Mexicano de Robótica (COMRob 2017). Mazatlán, Sinaloa, México: Asociación Mexicana de Robótica e Industria AC.

[3] Delgado-Báez, J. A., Castro-Linares, R., & Velasco-Villa, M. (2014). Formación de robots móviles omnidireccionales considerando su modelo dinámico. En Memorias del XVI Congreso Latinoamericano de Control Automático (CLCA 2014) (pp. 14-17). Cancún, Quintana Roo, México.

[4] Dellaert, F., & Sethi, S. (s. f.). *Differential drive robot actions*. Robotics from first principles. https://www.roboticsbook.org/S52_diffdrive_actions.html

[5] Klančar, G., Zdešar, A., Blažič, S., & Škrjanc, I. (Eds.). (2017). Wheeled mobile robotics: From fundamentals towards autonomous systems. Butterworth-Heinemann

[6] Blade. (2024, 2 octubre). *Understanding Odometry and Its Applications*. News - SparkFun Electronics. https://news.sparkfun.com/11361

[7] Dawkins, P. (2022, 16 noviembre). *Differential equations - Euler's Method*. Pauls Online Math Notes. https://tutorial.math.lamar.edu/classes/de/eulersmethod.aspx

[8]Howard, T. (s. f.). *Part 2: Odometry & Navigation*. ROS 2 Labs In The Diamond. Recuperado 17 de abril de 2026, de https://tom-howard.github.io/ros2/course/part2/

[9]*Odometry*. (s. f.). Nav_Msgs 4.9.1 Documentation. Recuperado 17 de abril de 2026, de https://docs.ros.org/en/humble/p/nav_msgs/msg/Odometry.html

[10]Ramírez Benavides, K. (s. f.). Odometría [Diapositivas de Powerpoint]. Recuperado 17 de abril de 2026, de https://www.kramirez.net/Robotica/Material/Presentaciones/Odometria.pdf